

モジュール Memory Module

蓬莱 尚幸

1. 概要

MIPS CPU シミュレータ制作のために、ビッグエンディアンかつワード境界・ハーフワード境界によるアラインメントの機能を持つ 32 ビットアドレスを持つ 8 ビットメモリを実現するためのモジュールである。

1.1 ファイル構成

- ・ ヘッダファイル：MemoryModule.h
本モジュールを利用するプログラムでは、必ず、include すること。
- ・ ソースコードファイル：MemoryModule.c
本モジュールは、(アーカイブ)ライブラリではなく、ソースコードとして提供する。

1.2 仕様

- ・ 定義されている型：
 - MADDR：メモリアドレスを表す型。
 - MEMORY：8 ビットメモリをあらわす型。
- ・ 定義されている関数：
 - MEMORY *MMalloc(size_t size)：メモリの動的な確保
 - void MMwriteWord(MEMORY *mem, MADDR addr, uint32_t data)：
32 ビットワードデータのストア。
 - uint32_t MMreadWord(MEMORY *mem, MADDR addr)：
32 ビットワードデータのロード。
 - void MMwriteHalfWord(MEMORY *mem, MADDR addr, uint16_t data)：
16 ビットハーフワードデータのストア。
 - uint16_t MMreadHalfWord(MEMORY *mem, MADDR addr)：
16 ビットハーフワードデータのロード。
 - void MMwriteByte(MEMORY *mem, MADDR addr, uint8_t data)：
8 ビットバイトデータのストア。
 - uint8_t MMreadByte(MEMORY *mem, MADDR addr)：

8 ビットバイトデータのロード。

- `void MMloadProgram(MEMORY *mem, MADDR addr, uint32_t code[], size_t len)` :
プログラム(命令列)のメモリへのロード。

2. 型の詳細

2.1 型 MEMORY

8 ビット(1 バイト)のメモリ 1 個を表す型である。本モジュールでは、メモリを MEMORY 型配列で実現することを仮定している。

2.2 型 MADDR

本モジュールで提供するメモリのアドレスを表す型である。本モジュールでは、メモリを MEMORY 型で実現することを仮定しているので、メモリのアドレスはその配列のインデックスである。また、MIPS アーキテクチャは 32 ビットアドレスである。そこで、メモリの先頭バイトのアドレスは 0 であり、可能な最大アドレスは $2^{32} - 1$ である。

3. 関数の詳細

3.1 関数 MMalloc

`MEMORY *MMalloc(size_t size);`

本モジュールが提供するメモリを動的に確保するための関数である。size で指定したバイト数だけメモリを確保し、獲得したメモリ(複数)へのポインタを返す。確保に失敗すると、エラーメッセージを標準エラー出力に表示し、異常終了する。

3.2 関数 MMwriteWord

`void MMwriteWord(MEMORY *mem, MADDR addr, uint32_t data);`

MEMORY 型配列 mem[] のアドレス addr(符号無し 32 ビット値)から始まる 4 バイトに 32 ビットデータ data をビッグエンディアンでストアする。addr が 4 の倍数でない(ワード境界ではない)場合、エラーメッセージを標準エラー出力に表示し、異常終了する。

3.3 関数 MMreadWord

```
uint32_t MMreadWord(MEMORY *mem, MADDR addr);
```

MEMORY 型配列 mem[] のアドレス addr(符号無し 32 ビット値)から始まる 4 バイトにビッグエンディアンで格納されている 32 ビットデータをロードし、uint32_t 型の値に変換して返す。addr が 4 の倍数でない(ワード境界ではない)場合、エラーメッセージを標準エラー出力に表示し、異常終了する。

3.4 関数 MMwriteHalfWord

```
void MMwriteHalfWord(MEMORY *mem, MADDR addr, uint16_t data);
```

MEMORY 型配列 mem[] のアドレス addr(符号無し 32 ビット値)から始まる 2 バイトに 16 ビットデータ data をビッグエンディアンでストアする。addr が 2 の倍数でない(ハーフワード境界ではない)場合、エラーメッセージを標準エラー出力に表示し、異常終了する。

3.5 関数 MMreadHalfWord

```
uint16_t MMreadHalfWord(MEMORY *mem, MADDR addr);
```

MEMORY 型配列 mem[] のアドレス addr(符号無し 32 ビット値)から始まる 2 バイトにビッグエンディアンで格納されている 16 ビットデータをロードし、uint16_t 型の値に変換して返す。addr が 2 の倍数でない(ハーフワード境界ではない)場合、エラーメッセージを標準エラー出力に表示し、異常終了する。

3.6 関数 MMwriteByte

```
void MMwriteByte(MEMORY *mem, MADDR addr, uint8_t data);
```

MEMORY 型配列 mem[] のアドレス addr(符号無し 32 ビット値)に 1 バイト=8 ビットデータ data をストアする。バイトデータのストアなので、境界チェックは行わない。

3.7 関数 MMreadByte

```
uint8_t MMreadHalfWord(MEMORY *mem, MADDR addr);
```

MEMORY 型配列 mem[] のアドレス addr(符号無し 32 ビット値)に格納されている 1 バイト=8 ビットデータをロードし、uint8_t 型の値として返す。バイトデータのロードなので、境界チェックは行わない。

3.8 関数 MMloadProgram

```
void MMloadProgram(MEMORY *mem, MADDR addr, uint32_t code[], size_t len);
```

32 ビット符号無し整数型配列 `code[]` (要素数 `len`) に格納されているデータを MEMORY 型配列 `mem[]` のアドレス `addr` (符号無し 32 ビット値) から始まる $4 \times \text{len}$ バイトに (各要素ごとに) ビッグエンディアンでストアする。`addr` が 4 の倍数でない (ワード境界ではない) 場合、エラーメッセージを標準エラー出力に表示し、異常終了する。一連のマシン語プログラムやデータをメモリ上にロードするために使うことができる。

以 上